

World Model Deep-Dive

“The image of the world around us, which we carry in our head, is just a model. Nobody in his head imagines the world, government or country. He has only selected concepts, and relationships between them, and uses those to represent the real system.”

Intro

Google launched [The Waymo World Model](#) to enable autonomous driving simulation. [World Labs](#) recently raised an additional \$1B from a who's-who of tech and finance — NVIDIA, AMD, Autodesk, Fidelity, and others. [General Intuition](#) raised \$134M seed to use world models to help Ukrainian soldiers simulate and navigate life-threatening battlefield scenarios.

Each represents a bet that machines can learn to model and simulate the physical world. So what's behind all the world model hype?

This analysis is written for a semi-technical audience looking to understand the general intuition and different types of world models. It tackles four key questions / topics:

1. **What is a world model?** A clear definition, taxonomy of different types of world models being developed today, and the research lineage behind each.
2. **Who's building them and what are they used for?** Comparison of current leading approaches — mapping use cases and applications across key companies in the space.
3. **What's holding them back?** Core technical bottlenecks across different approaches.
4. **Let's play.** Hands-on experiments across different types of world models and what we found.

Section 1: What is a world model?

Over the past two years, the term "world model" has been stretched almost beyond recognition — applied by researchers, companies, and VCs to everything from large language models to game engines to simple predictive simulators, regardless of what was actually under the hood.

Getting the definition right matters, because the type of world model determines what it can and can't do. In this section we share a brief history, a clear definition, and the research lineage behind each major type.

A Brief History

The idea of a world model predates modern AI entirely. In 1943, psychologist Kenneth Craik proposed that the human mind works by constructing small-scale models of reality to anticipate events — essentially, that thinking is a form of simulation. *Elon Musk has since claimed there's a billion to one chance we're not already living in one.*

But the engineering question is more grounded: can machines learn to simulate their world well enough to act intelligently within it? The concept re-emerged in reinforcement learning research through the 1990s and 2000s, where model-based RL agents learned internal representations of their environments to plan ahead without requiring exhaustive real-world interaction. The term “world model” was eventually coined by David Ha and Jürgen Schmidhuber in their 2018 research.

We've included an overview of historical waves in world models in the exhibit below. Note that the historical narrative is debated — this reflects the broad arc rather than a definitive account. For a more complete history, see Packy McCormick and Pim de Witte's [article](#).

Exhibit: World Model History in Waves

Wave	Years	Key Questions & Findings	Key Papers
Wave 0	1943	What defines “world” for us human? In 1943, psychologist Kenneth Craik proposed that the human mind works by constructing small-scale models of reality to anticipate events.	The Nature of Explanation — Craik (1943)
Wave 1	1990–1991	What would a world model do? Model-based RL agents learned internal representations of their environments to <u>plan</u> , <u>ahead</u> without exhaustive real-world interaction.	Dyna — Sutton (1991) Making the World Differentiable — Schmidhuber (1990)
Wave 2	2018–2019	Does this even work? Ha & Schmidhuber showed agents could learn compressed spatial/temporal representations and train controllers entirely inside a “dream” environment — proving world models work end-to-end.	World Models — Ha & Schmidhuber (2018) PlaNet — Hafner et al. (2019)
Wave 3	2020–2022	Can the world model match human performance? DreamerV2 became the first world-model agent to reach human-level performance across 55 Atari games on a single GPU. MuZero matched it via the opposite approach — planning with a purely implicit model.	DreamerV1/V2 — Hafner et al. (2020/2021) MuZero — Schrittwieser et al., Nature (2020) IRIS — Micheli et al. (2022) JEPA — LeCun (2022)
Wave 4	2023–2024	Can world models be truly interactive? Models moved beyond passive prediction toward real-time user control — generating driveable scenes, playable game environments, and action-conditioned video from text prompts.	Genie — Google DeepMind (2024) GAIA-1 — Wayve (2023) DIAMOND — Alonso et al. (2024) UniSim — Yang et al., ICLR (2024)
Wave 5	Present	Can world models enter the real world? World models are being deployed in physical AI systems — enabling robots, autonomous vehicles, and embodied agents to plan in open-ended real-world environments using video-pretrained backbones.	DreamerV3 — Hafner et al., Nature (2025) Cosmos — NVIDIA (2025) V-JEPA 2 — Assran et al., Meta (2025) Genie 3 — Google DeepMind (2025)

Definition

We define the term “world model” as an action-conditioned forward dynamics model. Given a state and action, the model can predict the next resulting state. Video generation models like Sora that only generate observations of the world but don't model the consequence of acting within it, is not the type of world model this paper focuses on.

Exhibit: Definition of World Model



World Model Taxonomy

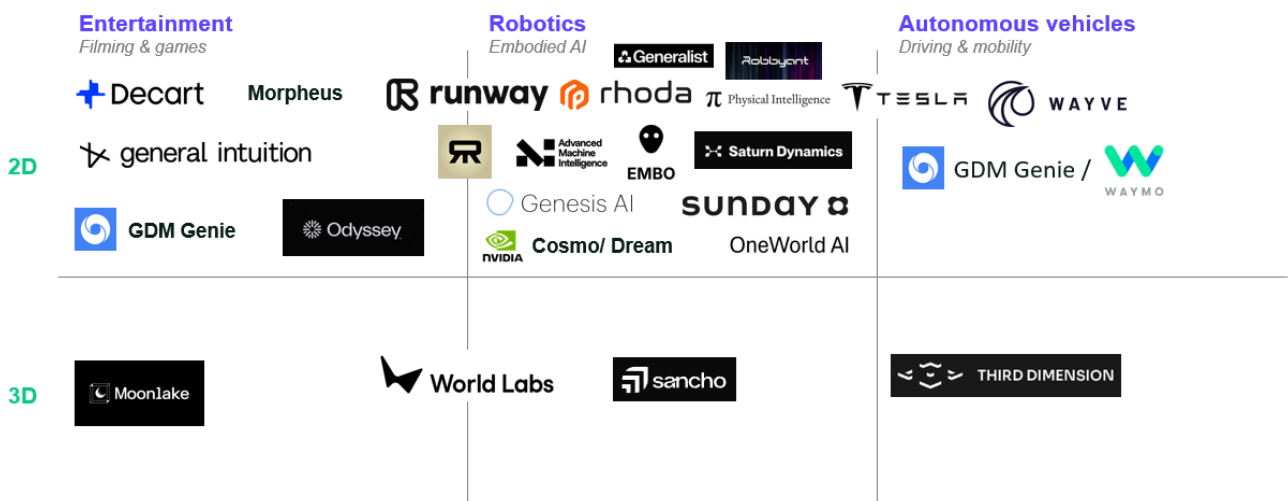
"World models" can be an overloaded term, making categorization challenging — though many have attempted it.

We view a useful taxonomy emerges along two axes:

1. **How does the model handle space?** (2D vs. 3D)
2. **What is the model's use case?** (Entertainment vs. Robotics vs. AV)

Exhibit: World Model Taxonomy along Two Axis

Mapped by use case (columns) and spatial dimensionality of the model (rows)



The remainder of this section focuses on how models handle modeled space. Section 2 covers applications and company use cases.

2D World Models

2D generative world models (e.g., Google's [Genie](#), Tesla's World Simulator, NVIDIA Cosmos), are the most intuitive entry point. **The core idea is straightforward: given what the world**

looked like up to this moment, and given the action just taken, predict what it looks like next — frame by frame, pixel by pixel. It's world modeling as video prediction.

Video generation vs. 2D world models — an important distinction

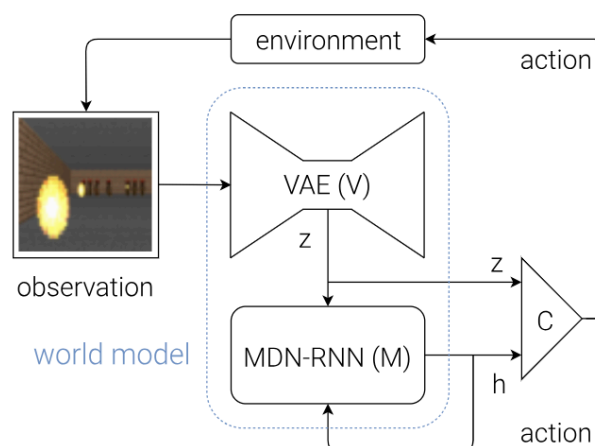
Before going further, it's worth drawing a line that the industry has frequently blurred. A video generation model produces a fixed artifact — a clip that plays from start to finish regardless of what a viewer does. Pika, Runway's Gen-4, and Luma, These are impressive generative systems, but they are not world models under our definition. They generate observations of the world without modeling the consequences of acting within it.

A 2D generative world model produces an environment — one that responds to actions, maintains state over time, and evolves differently depending on what an agent does inside it. The output isn't a clip to watch; it's a space to act in. The defining characteristic is **interactivity**. Remove it, and you have a video generator. Keep it, and you have a world model.

This **lineage** traces directly back to Ha & Schmidhuber's 2018 [World Models](#) paper, which first demonstrated an agent could learn inside its own imagination. The paper proposed that an agent could be split into three parts:

- A **visual encoder** (V) that compresses raw pixel observations into a compact representation, like how your eye doesn't send every photon to your brain, but a compressed summary.
- A **memory model** (M), a recurrent neural network (that predicts what comes next, essentially the world model itself).
- A **controller** (C), a small, simple decision-maker that acts based on V and M's outputs.

Exhibit: World Model Architecture



Flow diagram of our Agent model. The raw observation is first processed by V at each time step t to produce z_t . The input into C is this latent vector z_t concatenated with M's hidden state h_t at each time step. C will then output an action vector a_t for motor control. M will then take the current z_t and action a_t as an input to update its own hidden state to produce h_{t+1} to be used at time $t + 1$.

What changed in the years that followed was the quality of the underlying generative machinery. Two distinct model architectural approaches emerged to solve the same core problem (i.e., predicting the next frame conditioned on an action) and they make meaningfully different tradeoffs.

1. **Autoregression generation:** autoregressive model along spatial dimension — model generates each frame by predicting spatial tokens in order (patch 1, then patch 2, then patch 3... within that frame). Google's Genie takes this approach — using a transformer that conditions on past frames and actions to generate what comes next. These models tend to be faster at inference and better at maintaining temporal consistency¹ across frames.
2. **Diffusion generation:** generate the next frame by starting with a blurry, noisy image and progressively refining it, step by step — until a sharp, coherent frame emerges. Think of it like a photograph developing in a darkroom: you start with nothing recognizable and gradually sharpen toward the final image. The model learns to guide this refinement process so the output matches what the world should look like given the current context and action. Google's [GameNGen](#) and [DIAMOND](#) take this approach, and Decart's [Oasis](#) combines both in a diffusion transformer. Diffusion models tend to produce higher visual fidelity and handle uncertainty more gracefully — but were long considered too slow for real-time interactive use. GameNGen and DIAMOND were the first to demonstrate this wasn't an inherent limitation, just an engineering one.

The field hasn't settled on a single approach. The most recent models, like Oasis, are beginning to combine both, suggesting the distinction may matter less going forward than it does today.

2D generative world models are not just the most intuitive — they are currently the most capable and widely deployed, for a few compounding reasons. The most important is data. Training a 2D world model requires video paired with actions — comparatively easier to collect than approaches requiring specialized data (e.g., 3D). Billions of hours of humans driving, playing games (e.g., [General Intuition](#) spun off from [Medal](#), a platform for uploading and sharing video game clips) already exist and can be repurposed directly. Legibility is another advantage. Because the outputs are visual, 2D world models are easy to evaluate, demo, and iterate on — accelerating research and commercial development in ways more abstract approaches can't match.

That said, they carry a structural ceiling. With no explicit representation of geometry or physics, predicting appearances rather than structure, errors accumulate across frames, temporal consistency degrades over time, and compute costs are high. We will return to these technical bottlenecks in detail in Section 3.

¹ Temporal consistency refers to a model's ability to maintain coherent, stable elements across consecutive frames over time (e.g., a wall stays where it was, an object doesn't change shape mid-scene, physics doesn't quietly break down). We go over this concept as a core technical bottleneck in Section 3.

Latent Space World Models

Latent world models is a technique that compresses observations into compact abstract representations and predicts how those representations evolve over time given different actions. All world models in practice have internal latent representations – the difference is not one of kind, but of supervision: whether you compute losses on semantic outputs only ("latent"/JEPA) or compute reconstruction losses on raw observations (e.g., pixel).

This is not a binary design choice — for example, a model can predict semantic outputs at every step while computing reconstruction losses only every 10 steps. The primary advantage of latent models is efficiency: by skipping pixel reconstruction, they are significantly faster to train and run.

Several schools of thought have emerged around latent space models, spanning reinforcement learning, latent planning with learned dynamics, and energy-based approaches — each laid out with key applications and notable papers in the table below. These categories are not clean separations; the lineages are deeply intertwined, and the divisions reflect ideology as much as methodology. Ultimately, they often come down to which problems a researcher prioritizes: agents, robotics, and control versus more general world modeling.

Exhibit Latent Space Models - Three Schools of Thoughts & Related Papers

	Overview	Application	Papers
Reinforcement learning	-Agents learn by practicing entirely inside an imagined world model rather than the real environment. -The architecture: RSSM (Recurrent State Space Model) maintains a compressed hidden state that updates step by step as the agent acts. -Built for action-first: the world model exists to develop better instincts, not to understand the world for its own sake.	Continuous control tasks (e.g., robotics)	Dyna (1990), SimPLe (2019), PlaNet (2019), Dreamer V1-V4 (2020-2025)
Latent planning with invented dynamics lineage	-Rather than developing instincts through imagination, the agent deliberates at every single move — consulting its world model to simulate possible futures and pick the best one. -The architecture invents an abstract latent space optimized purely for planning, not for resembling reality.	Any domain with a structured, well-defined action space where exhaustive look-ahead and precise planning outperform learned instincts	MuZero (2020), EfficientZero (2021), EfficientZero V2 (2024)

	-Built for discrete, structured problems where exhaustive look-ahead pays off.	(e.g., Board games)	
Energy based lineage	-Rather than generating pixel outputs, the model scores compatibility between predictions and reality in latent space — learning meaning rather than appearance. -Architecture: three components (context encoder, predictor, target encoder) with no pixel decoder. -Built for understanding-first: action-conditioning was added later as an extension.	Visual understanding, video prediction, representation learning for downstream tasks (e.g., robotics)	JEPA (2022), I-JEPA (2023), V-JEPA (2024), LeWorldModel (2026)

3D World Models

3D-based generative world models treat the world as a navigable space — one with depth, volume, and geometry. Their core objective: reconstruct 3D structure from 2D images captured across different angles. The result is consistent navigable 3D environments: objects stay where you left them, rooms look the same when revisited, and spatial relationships hold across long horizons.

Two most prominent methods used include *NeRF (Neural Radiance Fields)* and Gaussian Splatting. [NeRFs \(Mildenhall et al., 2020\)](#) encodes a 3D scene inside a neural network – given any position in space and any viewing direction, the network predicts what color and density exists there, allowing photorealistic images to be rendered from any angle, including angles never seen during training.

[Gaussian Splatting \(Kerbl et al., 2023\)](#) achieves the same goal differently. A Gaussian is simply a smooth blob — mathematically it's a bell curve, but in 3D space it's an ellipsoid. Think of it like a cloud of smoke: densest at the center, fading out smoothly toward the edges. Rather than encoding the scene inside a network, it represents it explicitly as a collection of millions of 3D blobs, each with a position, size, color, and opacity. Rendering is just projecting and blending those blobs onto the image plane — no network queries needed, which is why it runs in real time while NeRF takes minutes.

The limitation of 3D world models is structural, which we will cover more in section 3.

Note: *the taxonomy above is our attempt at categorization and intended for readers new to world models. In practice, categorizing world models is not straightforward — most real systems combine architectural approaches, and the boundaries between them are fluid. If you are curious about different attributes and how others have categorized world models, there are*

many survey papers out there worth taking a look (E.g., [A Step Toward World Models: a Survey on Robotic Manipulation](#)).

Section 2: Who's building world models and what are they being built for?

Section 1 established what world models are and how they work. This section maps where they're being deployed — the earliest commercial applications taking shape and the technical approaches different players are betting on.

Competitive Landscape [here](#)

Filming

One of the early commercial bets for world models is film and video production. The best example here is [World Labs](#). World Labs' first product [Marble](#), is a 3D World Model using Gaussian Splatting techniques discussed in Section 1. The architectural choice is deliberate: film production demands geometric consistency in a way that 2D generative approaches can't guarantee. A director needs to return to the same set from a different angle and have it look identical. 3D representations make that promise; pixel prediction models don't. The current limitation is dynamism — static architectural environments are well within reach, but simulating crowds, water, fire, or cloth at production quality remains an open problem. A case study with [HTC VIVE](#)'s virtual production system demonstrated this efficiency: three creators with no 3D experience generated over ten unique 3D environments, from futuristic interiors to moody cityscapes and filmed social content inside them in a single afternoon, a task that would traditionally require weeks of modeling and rendering work.

Gaming

Gaming is one of the clearest examples of the data flywheel dynamic in world models — the more users play, the more interaction data is collected, and the better the model gets. Two companies in this space, Moonvalley and General Intuition, are pursuing meaningfully different strategies:

- **Moonlake AI** builds the world model as the product. It focuses on reducing gaming development costs with its product, **Reverie**. This system generates interactive 3D environments from natural language prompts. Technically, it achieves persistence, where edits like a crumbling wall hold across frames – by conditioning the model on 3D structural signals rather than just pixel sequences. Case Study of MoonLake's Game Vibe Coding: [Here](#)
- **General Intuition** employs a data-centric strategy, leveraging its spinout from [Medal](#), which captures 2 billion gameplay clips annually. The near-term product is **AI-powered NPCs**. The longer-term bet is that games are the most effective environment for AI to learn to understand the real world, as they provide a verifiable domain for spatial-temporal reasoning where every action can be objectively assessed.

Autonomous Driving

Autonomous driving presents the "long tail" challenge: the rare, unpredictable, and dangerous scenarios that cause accidents are nearly impossible to gather training data for in the real world. World models solve this by simulating the extreme tail end of reality at scale (E.g., floods, wrong-way drivers, elephants crossing the street). We've examined two companies around how they use world models: [Tesla](#) and [Waymo](#).

Tesla

Tesla utilizes a world model primarily for "eval"—acting as a high-fidelity, neural simulator to stress-test driving policies prior to real-world deployment. Tesla's world model uses state-action data to fine-tune its E2E foundation model that's trained for driving. The world model allows Tesla to generate a full closed-loop simulation: the policy acts, the world model predicts what happens next, the policy responds to that prediction, and the loop continues. This allows Tesla to evaluate policy behavior across a vast range of scenarios without physical hardware, such as

- **Real world replay:** Resimulating historical edge cases to evaluate how a new policy build would behave.
- **Counterfactual Testing:** Altering variables (e.g., changing weather to a wet road, introducing a sudden cut-in, or blocking a lane) to test hundreds of parallel futures.
- **Extreme Stress-Testing:** Generating rare, hazardous edge cases that seldom occur on public roads.
- **Risk-Free Iteration:** Improving safety profiles at a massive scale without exposing drivers or pedestrians to physical risk.

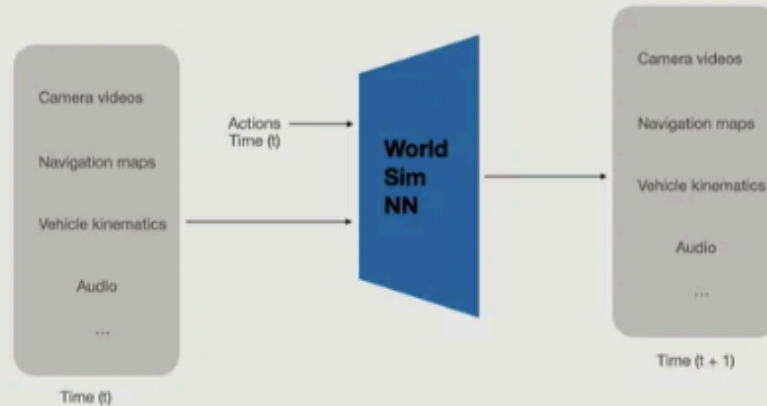
Furthermore, Tesla is generalizing this exact neural simulation framework to **Optimus**, allowing the humanoid robot to simulate and evaluate its physical interactions with the world. Note that [Wayve](#)'s approach resembles Tesla's.

Exhibit – How Tesla trains and uses world model for AV and Robotics

3. Evaluation

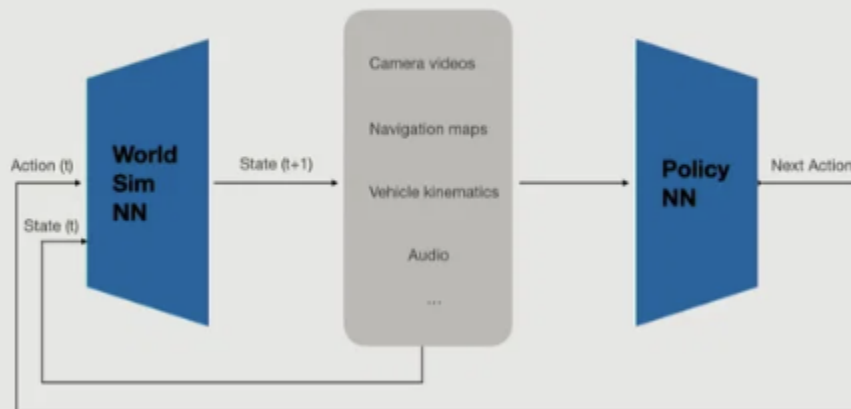
Neural Network closed-loop simulator

Can be trained with cheap to collect state-action data



3. Evaluation

Can then be used as a closed loop simulator

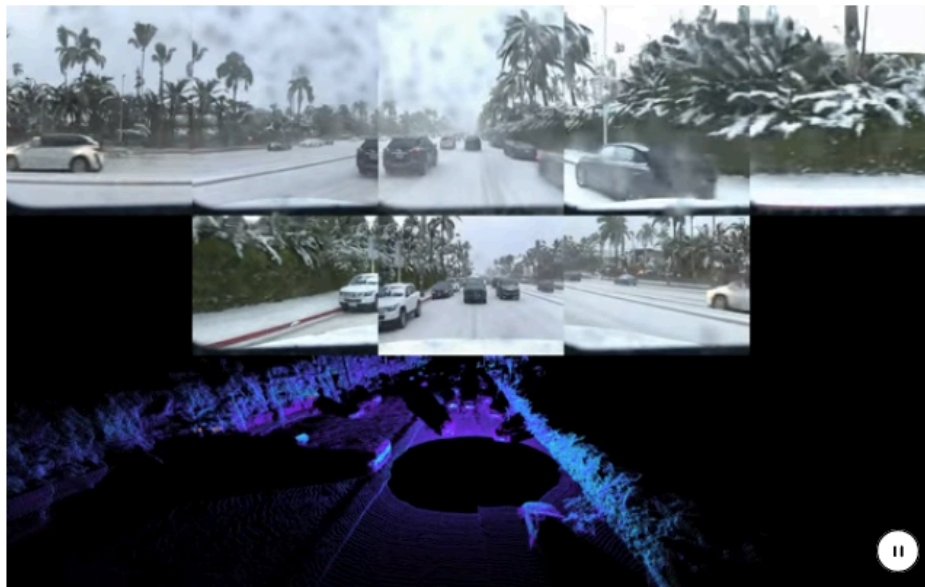


Source: [Building Foundational Models for Robotics at Tesla](#)

Waymo

In February 2026, Google published the Waymo World Model that's built on top of Google DeepMind's [Genie 3](#). Waymo's use case is more or less similar to Tesla's – using world models for training and evaluation to explore long tail of situations that are never / rarely observed by its fleet.

What's different is modality: Waymo simulates camera + 3D lidar + HD maps using post training from 2D video.



Simulation: Driving on a street with lots of palm trees in a tropical city, strangely covered in snow.

Robotics

Robotics and autonomous driving share more than intuition suggests — they both require an agent to perceive a physical environment, model what happens next, and act accordingly. Tesla makes this connection explicit: the same world model architecture underlying FSD is being transferred directly to its Optimus humanoid robot, a bet that the dynamics of navigating roads and navigating physical spaces are similar enough to share a foundation.

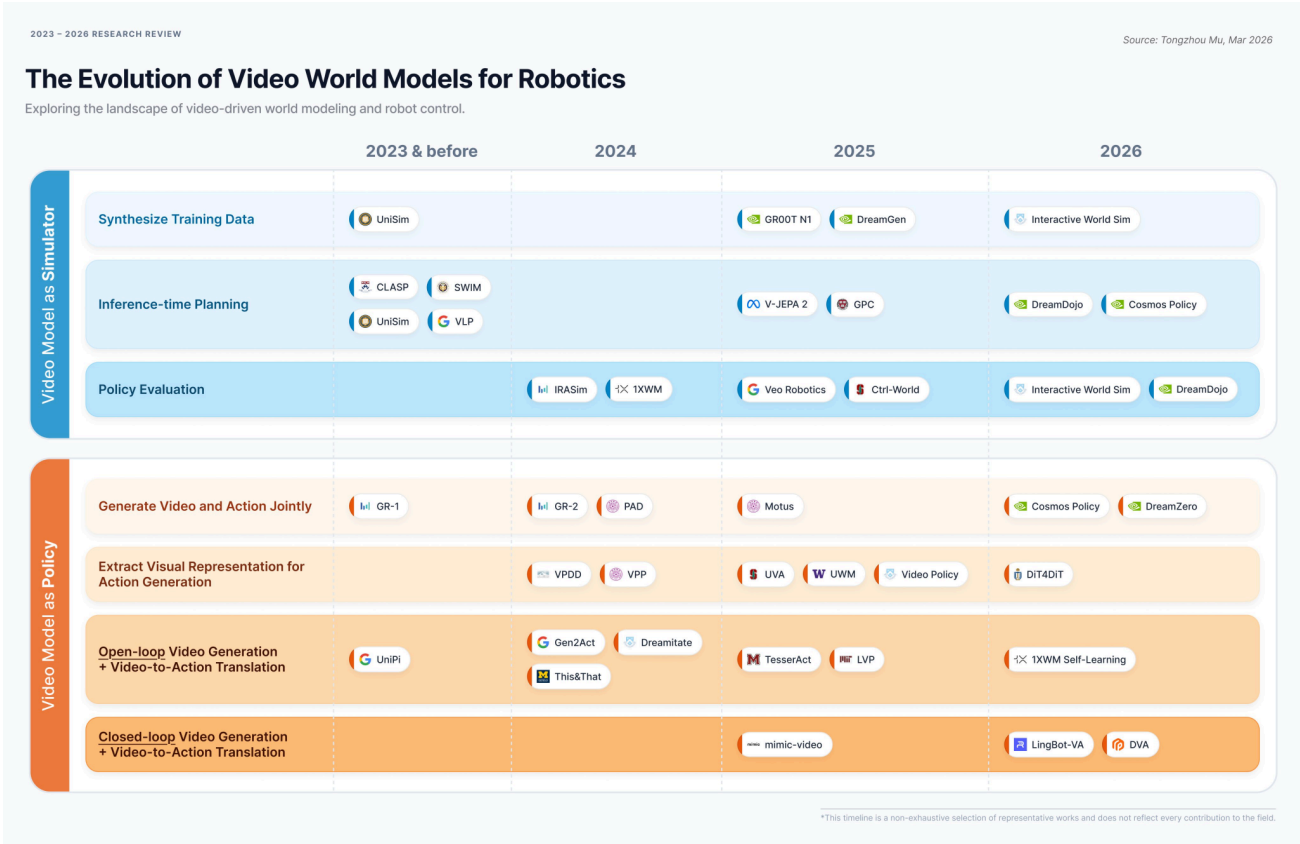
But robotics introduces challenges that autonomous driving doesn't face. A car operates in a relatively constrained environment (e.g., roads, lanes). A robot operating in an unstructured environment faces contact-rich manipulation, deformable objects, and unpredictable human interaction. Picking up a coffee cup, folding laundry, assembling a component — these require a world model that understands fine-grained physical dynamics, not just navigation. World models address this by giving robots the ability to reason about physical consequences before acting — two use cases are currently emerging:

- **World model as a simulator:** this approach uses world models to replace traditional physics simulators like IsaacSim, MuJoCo, and ManiSkill. Traditional simulators are hand-engineered — researchers manually define physics rules, object properties, and environment dynamics. They work well for structured, predictable tasks but struggle with the messiness of the real world: deformable objects, complex contact dynamics, and the infinite variety of unstructured environments a robot might encounter. A learned world

model sidesteps this by inferring inverse dynamics from real-world data. This unlocks four concrete capabilities:

- *Cheaper data generation.* Given a starting frame and a language instruction ("pick up the cup"), the world model generates synthetic video clips of a robot completing the task. An inverse dynamics model then automatically labels those clips with actions, producing robot policy training data at a fraction of the cost of physical collection. [DreamGen](#) and [GR00T N1](#) use this approach.
- *Inference-time planning.* Rather than committing to a single action, the robot simulates several possible futures inside the world model and selects the most promising one. The main challenge is latency — the simulation must run fast enough for real-time control.
- *Policy evaluation.* Before touching physical hardware, a robot policy can be stress-tested inside the world model — including adversarial scenarios designed to surface safety failures early.
- *Learning dynamics of the real world that are hard to simulate:* Rather than using traditional simulators, [DayDreamer](#), [Robotic World Model](#) papers take a different approach. Instead of hand-coding physics rules, they learn the dynamics of the real world directly from real robot interactions. Like Dreamer, they operate in latent space — but they don't predict images, only the physically relevant next state (joint positions, forces, contact signals). The result is a world model grounded in how the real world actually behaves, not how a simulator approximates it.
- **World model as a policy:** Instead of using the video model just to train or evaluate a separate controller, you use it as the controller itself. This usually uses an action-conditioned world model (e.g., [V-JEPA 2](#), [DreamZero](#)). The robot's decision-making is baked directly into the video model. Because the native outputs are videos rather than robot actions, several methods have been developed to obtain control signals.
 - *Generate video and actions simultaneously.* The simplest approach: take a video model and bolt on an action decoder alongside it. The model generates video frames and robot actions at the same time, in one pass.
 - *Use the video model as a feature extractor, not a generator.* Rather than generating full video, you only use the video model's internal representations. The video model never actually produces a video. The advantage is speed — you skip the expensive generation step, making real-time control more feasible.
 - *Generate a desired future video, then translate to actions (open-loop).* Rather than generating actions directly, the model first generates a video of what success looks like — "here is what the robot completing this task should look like" — and then a separate inverse dynamics model watches that video and figures out what actions would produce it.
 - *Generate a desired future video, then translate to actions (closed-loop)* Same idea as open-loop, but the model constantly updates its generated video based on what's actually happening in the real world. Rather than committing to one

blueprint upfront, it's like an architect who checks the building site after every wall goes up and revises the plans accordingly.



Scientific Simulation

Scientific simulation is perhaps the highest-stakes application of world models — and the least discussed. Traditional simulation requires researchers to already know the rules: write down the physics equations, then compute forward. World models offer a different path: learn the dynamics directly from experimental data, without needing to know the underlying equations first. This has meaningful implications across molecular dynamics, climate modeling, materials discovery, and nuclear fusion — domains where the cost of physical experimentation is enormous and the value of faster, cheaper simulation is correspondingly high.

Most of these applications remain research-stage or narrow deployments — the general-purpose scientific world model doesn't exist yet. But the early signals are compelling enough to watch closely.

Section 3: Core technical bottlenecks across different approaches.

2D Generative Model

Video-centric world models are impressive in short bursts. Genie's beta program (e.g., Project Genie), for instance, can sustain coherent interactive environments for up to 60 seconds.

However, over longer horizons, they suffer from **spatiotemporal inconsistency**: the model's internal representation of the world gradually drifts from coherence. This manifests as failures of *object permanence*, where items disappear or change properties mid-scene; *spatial drift*, where a room looks different when you retrace your steps; and *violations of basic causal dynamics*, where objects pass through surfaces or liquids ignore gravity.

The deeper technical issue is **compounding rollout error (model drift)**. World models simulate the consequences of an agent's actions by chaining predictions: the model imagines step 1, predicts step 2 from that, step 3 from step 2 — and errors accumulate exponentially. By step 10 of a robot manipulation task, the imagined world bears little resemblance to reality.

A second structural limitation is **control latency** that is caused by architectural inheritance. Models like Genie 3 are fine-tuned from general-purpose video generation models — in Genie 3's case, built on top of Veo 3. This foundation optimizes for visual plausibility over physical accuracy, which creates a ceiling for applications requiring precise real-time interaction. This issue hits hard for robot training specifically, where a model needs to faithfully simulate fine-grained real-time contact dynamics (e.g., Genie has a reported action-to-render latency of ~1.8 seconds).

The third limitation is **compute cost**. Current 2D generative world models must regenerate every pixel from scratch at each step — and to maintain temporal consistency, they condition on an expanding window of past frames. The cost compounds as the sequence grows: a model simulating 100 steps is doing dramatically more work than one simulating 10, which makes long-horizon simulation increasingly expensive and creates a hard practical ceiling for real-time applications.

3D Generative Model

The core strength of 3D world models is their explicit **geometric representation** — rather than generating pixels from scratch at every timestep, they ground generation in a persistent 3D scene scaffold. The result is that object identity and spatial layout are preserved by construction, not learned statistically. Objects stay where you left them. Rooms look the same when revisited. Basic physical constraints (e.g., occlusion, depth, surface continuity) hold across long horizons without needing to be re-derived from pixels at every step. This also makes them significantly cheaper to run than 2D generative models.

The tradeoffs are:

- 1) 3D world models **can't handle dynamic content**: these models are designed for static scenes and struggle with dynamic content (e.g., water flowing, complex deformations), the type of interaction that matters for physical manipulation and general-purpose simulation.
- 2) Even if the model can handle dynamic content, it still **wouldn't understand physical causality** (geometry vs. physics problem): A Gaussian splat encodes where things are and what they look like — it does not encode how they behave when forces are applied. The explicit structure that solves the consistency problem simultaneously prevents the model from internalizing the physical dynamics that make a simulator useful. This limitation cuts across both 2D and 3D world models — neither approach guarantees understanding of physical causality.
- 3) **Data scarcity**: 3D representations require multi-view video or depth data (e.g., LiDAR) for training, which is expensive and technically demanding to collect at scale. The richness and environmental variety achievable with internet-scale 2D video simply isn't available in 3D, limiting current models to narrow, curated domains (e.g., driving scenes, indoor rooms, specific object categories).

Latent Space Model

Latent world models sidestep the rendering constraints that limit both 2D and 3D approaches — no pixel generation, no geometric scaffolding, just compressed abstract representations of what matters. This makes them significantly cheaper to run and architecturally flexible in ways the other two approaches aren't.

The core tradeoff is that compression is always lossy. When an observation is encoded into a lower-dimensional latent state, information is discarded — and the model chooses what to discard based on what it learned during training, not necessarily what matters for the task at hand. A model might faithfully preserve texture and color while silently dropping the subtle structural cues that are actually decision-critical: the slight lean of an unstable tower, the trajectory of a moving object, the shadow that signals an obstacle just outside the frame. These details may be imperceptible to a human glancing at a scene but essential to an agent acting within it.

Section 4: Experiments across world models

Toggle:

- Tasks – 3 prompts
- Model providers – World Lab Marble, Google Project Genie, [Moonlake, Odyssey]

Overview:

Even though there are a lot of World Model companies out there, the actual products that are available for testing / prototyping are limited. We have tried the models that are either 1) now already available, 2) responded to our inquiry.

Prompt #1: Brutalist Library Golden Hour

Rationale: Tests geometric consistency, lighting fidelity, and material rendering — core strengths of 3D approaches. Reveals whether 2D models maintain consistent shadows and reflections from different angles.

- Marble: <https://marble.worldlabs.ai/world/6de100b3-6534-4be4-b99e-6f780f470779>
- Project Genie:
<https://labs.google/fx/projectgenie/tools/projectgenie/692060b4-aecf-4e79-ab4e-5271413f62f9>

Prompt #2: A coastal cliffside at dusk during a storm. Waves crash against the rocks below, sea spray rises, and tall grass bends in the wind. A narrow dirt path runs along the edge

Rationale: Targets the dynamic content weakness of 3D models — water, wind, grass movement. Tests whether each platform can handle multiple simultaneous dynamic elements or freezes them into a static scene.

- Marble: <https://marble.worldlabs.ai/world/d91117e9-fae9-4dc3-b712-49763db64b3b>
- Project Genie:
<https://labs.google/fx/projectgenie/tools/projectgenie/3e4ce51d-1753-48d9-911e-6b1cff840b27>

Prompt #3: *A cluttered workshop with tools hanging on pegboards, a half-assembled motorcycle on a workbench, oil stains on the concrete floor, and a single bare bulb overhead.*

Rationale: Tests object diversity, spatial layout coherence, and fine-grained detail rendering. Also a good stress test for navigability — can you move around the motorcycle without geometry breaking down?

- Marble:
- Project Genie:
<https://labs.google/fx/projectgenie/tools/projectgenie/5abdfb77-3f38-4412-9f3f-6d80ab0f1402>